

AfyaFlow AI Build Journey

Purpose of This Document

This document structures the AfyaFlow AI project as both a software build plan and a LinkedIn documentation journey.

The goal is to help the builder move from idea to deployed MVP while documenting the full process publicly in a professional, product-focused way.

The journey should show three things:

1. Product thinking
2. Technical execution
3. Consistency in shipping

The LinkedIn positioning should not be about being a beginner. It should be about building in public, learning in public, and showing progress through a real product.

1. Project Overview

Project Name

AfyaFlow AI

Product Description

AfyaFlow AI is an AI-powered healthcare assistant designed to support patients and clinics by helping users ask routine health questions, book appointments, receive reminders, and access basic healthcare guidance.

The product is not intended to replace healthcare professionals. It is designed to support patient engagement, reduce repetitive administrative workload, and guide users toward appropriate care when needed.

Product Tagline

AI-powered patient support for smarter clinic workflows.

Target Users

Primary Users

Patients who need basic health information, appointment support, and reminders.

Secondary Users

Clinics, clinical officers, nurses, and healthcare administrators who want to reduce repetitive patient inquiries and improve patient follow-up.

2. Recommended Technology Stack

Backend

Python with Django

API Layer

Django REST Framework

Database

PostgreSQL

Frontend Option 1

Django templates with Tailwind CSS for a simple MVP

Frontend Option 2

React frontend connected to Django REST APIs for a more advanced version

AI Integration

OpenAI API or Claude API through a service layer that can be changed later

Development Productivity Tool

Claude Code for accelerated development, code review, debugging, test generation, documentation, and refactoring

Deployment

Frontend and backend can be deployed using Render, Railway, Fly.io, or a similar platform that supports Django and PostgreSQL.

Version Control

Git and GitHub

3. Why This Stack Works

Django is a good fit because it gives the project a strong backend foundation quickly. It includes authentication, admin dashboards, models, forms, routing, security features, and a mature ecosystem.

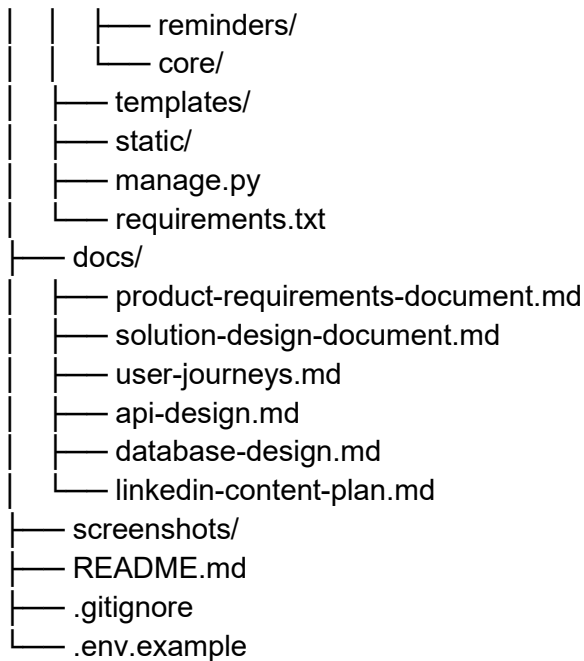
PostgreSQL is suitable because healthcare-related applications need structured data, reliability, relational models, and future reporting capabilities.

Django REST Framework makes it easier to expose APIs for chatbot messages, appointments, reminders, and future mobile app integrations.

Claude Code can help speed up the development process by turning requirements into code tasks, generating boilerplate, explaining errors, writing tests, reviewing code, and improving documentation.

4. Suggested Project Folder Structure

```
AfyaFlow-AI/
├── backend/
│   ├── afyaflow/
│   │   ├── settings.py
│   │   ├── urls.py
│   │   ├── asgi.py
│   │   └── wsgi.py
│   └── apps/
│       ├── accounts/
│       ├── chatbot/
│       └── appointments/
```



5. Week 1: Product Foundation, PRD and Solution Design

Goal

Define the product properly before writing too much code.

Week 1 should not only be about setting up folders. It should create the documents that guide the whole build.

The key deliverables are:

1. Product Requirements Document
 2. Solution Design Document
 3. User journeys
 4. Initial database thinking
 5. GitHub repository setup
 6. Claude Code development workflow
-

Week 1 Deliverable 1: Product Requirements Document

The PRD should explain what AfyaFlow AI is, why it matters, who it serves, and what the first MVP should include.

PRD Structure

1. Product Name
2. Product Vision
3. Problem Statement
4. Target Users
5. User Pain Points
6. MVP Scope
7. Out of Scope for Version 1
8. Functional Requirements
9. Non-Functional Requirements
10. User Journeys
11. Success Metrics
12. Risks and Assumptions
13. Future Enhancements

Sample Product Vision

AfyaFlow AI aims to improve patient support by providing a digital assistant that helps users access general health guidance, book appointments, and receive reminders while encouraging safe and timely engagement with healthcare professionals.

Sample Problem Statement

Many clinics receive repeated patient inquiries about symptoms, appointments, follow-ups, medication reminders, and general health questions. This creates workload for healthcare workers and delays patient support. AfyaFlow AI provides a simple digital channel to handle routine support while directing patients to professional care when needed.

MVP Scope

The first version should include:

1. Home page
2. Patient chatbot
3. Appointment booking form
4. Reminder creation form
5. Basic admin view
6. Chat safety rules
7. PostgreSQL database
8. Deployed demo

Out of Scope for Version 1

The first version should not include:

- 1. Final medical diagnosis
- 2. Prescription generation
- 3. Emergency medical handling beyond guidance to seek urgent care
- 4. Insurance claims
- 5. Payment integration
- 6. Hospital system integration
- 7. Patient medical record integration

Week 1 Deliverable 2: Solution Design Document

The Solution Design Document should explain how the system will be built.

Solution Design Document Structure

- 1. System Overview
- 2. Architecture Diagram
- 3. Technology Stack
- 4. Application Modules
- 5. Database Design
- 6. API Design
- 7. AI Integration Design
- 8. Security Considerations
- 9. Deployment Design
- 10. Testing Approach
- 11. Risks and Mitigations
- 12. Future Scalability

Application Modules

Module	Purpose
Accounts	Manage users and authentication in future versions
Chatbot	Handle patient conversations with the AI assistant
Appointments	Allow users to request clinic appointments
Reminders	Allow users to create appointment or medication reminders

Core	Shared utilities, homepage, base templates, and common logic
Admin	Use Django admin to manage data during MVP

High-Level Architecture



Database Entities

Entity	Purpose
User	Stores basic user details where authentication is enabled
PatientProfile	Stores optional patient profile information
ChatSession	Groups user conversations
ChatMessage	Stores user and AI messages
Appointment	Stores appointment requests
Reminder	Stores reminder details

Week 1 Deliverable 3: Claude Code Productivity Workflow

Claude Code should be used as a development accelerator, not as a replacement for understanding the code.

The recommended workflow is:

1. Write the requirement first
2. Ask Claude Code to propose the implementation plan
3. Review the plan
4. Ask Claude Code to generate or modify code
5. Run the app locally
6. Test the feature manually
7. Ask Claude Code to write tests
8. Review the tests
9. Commit the working change to GitHub
10. Document the lesson on LinkedIn

Example Claude Code Prompts

Read the PRD and Solution Design Document in the docs folder. Propose the Django apps, models, routes, and templates needed for the MVP. Do not edit files yet. First give me the implementation plan.

Create the Django app structure for chatbot, appointments, reminders, accounts, and core. Follow the architecture described in docs/solution-design-document.md.

Review my Django models for Appointment, Reminder, ChatSession, and ChatMessage. Suggest improvements for naming, relationships, validation, and future scalability.

Write unit tests for the appointment booking flow. Include tests for successful creation, missing required fields, invalid date, and API response format.

Review the codebase and generate a clean README that explains setup, environment variables, database setup, running migrations, and testing.

Week 1 Technical Tasks

1. Create GitHub repository
 2. Create Django project
 3. Configure PostgreSQL
 4. Create `.env.example`
 5. Create docs folder
 6. Write PRD
 7. Write Solution Design Document
 8. Create initial README
 9. Create first GitHub commit
 10. Use Claude Code to review project structure
-

Week 1 LinkedIn Post

Post Theme

Starting the AfyaFlow AI build journey with product thinking before code.

Sample LinkedIn Post

I'm starting a new build journey: AfyaFlow AI.

AfyaFlow AI is an AI-powered healthcare assistant designed to support patients and clinics through routine health guidance, appointment booking, and reminders.

Before jumping into code, I'm starting with product documentation.

This week, I'm working on:

- Product Requirements Document
- Solution Design Document
- User journeys
- Database planning
- GitHub repository setup
- Django and PostgreSQL project setup

The goal is to build this like a real product, not just a coding exercise.

I'm also using Claude Code as part of my development workflow to help with planning, code generation, debugging, tests, and documentation. The important part is to use AI to move faster while still understanding every decision.

Tech stack:

- Python
- Django
- Django REST Framework
- PostgreSQL
- Tailwind CSS
- AI API integration

I'll be documenting the full journey from idea to deployed MVP.

#BuildInPublic #HealthTech #AI #Django #Python #PostgreSQL #SoftwareEngineering

6. Week 2: Backend Foundation and Database Models

Goal

Build the backend foundation and database structure.

By the end of Week 2, the Django project should have working apps, models, migrations, admin setup, and basic APIs.

What to Build

Django Apps

1. Core
2. Chatbot
3. Appointments
4. Reminders
5. Accounts

PostgreSQL Models

Appointment Model

Fields:

1. Patient name
2. Phone number
3. Email address
4. Preferred date
5. Preferred time
6. Reason for visit
7. Status
8. Created at
9. Updated at

Reminder Model

Fields:

1. Patient name
2. Reminder type
3. Reminder message
4. Reminder date
5. Reminder time
6. Status
7. Created at

ChatSession Model

Fields:

1. Session ID
2. User reference where applicable
3. Started at
4. Last active at

ChatMessage Model

Fields:

1. Chat session
2. Sender type
3. Message content
4. Safety category
5. Created at

APIs to Build

Feature	Method	Endpoint
Create appointment	POST	/api/appointments/
List appointments	GET	/api/appointments/
Create reminder	POST	/api/reminders/
List reminders	GET	/api/reminders/
Create chat message	POST	/api/chat/
Get chat history	GET	/api/chat/sessions/{id}/

Claude Code Tasks for Week 2

Use Claude Code to:

1. Generate model drafts
2. Review model relationships
3. Create serializers
4. Create API views

5. Create URL routes
6. Write unit tests
7. Review Django admin setup
8. Refactor repeated logic

Example Prompt

Using the Solution Design Document, create Django models for Appointment, Reminder, ChatSession, and ChatMessage. Add clear field names, timestamps, status choices, and admin registration. Explain the design choices after making the changes.

LinkedIn Post for Week 2

Post Theme

Turning product requirements into backend structure.

Sample LinkedIn Post

Week 2 of building AfyaFlow AI.

This week, I focused on the backend foundation.

After creating the Product Requirements Document and Solution Design Document, I started translating the product into Django apps, database models, and API endpoints.

I worked on:

- Django project setup
- PostgreSQL configuration
- Appointment model
- Reminder model
- Chat session model
- Chat message model
- Django admin setup
- Initial API routes

One of the biggest lessons this week was that good backend development starts with clear product thinking.

If the requirements are unclear, the database design becomes messy.

If the user journeys are unclear, the APIs become confusing.

I'm also learning how to use Claude Code more intentionally. Instead of asking it to simply write code, I'm using it to review my thinking, suggest structure, generate tests, and explain tradeoffs.

Next step: building the user interface and connecting it to the backend.

#BuildInPublic #Django #Python #PostgreSQL #BackendDevelopment #HealthTech #AI

7. Week 3: User Interface and AI Chatbot Integration

Goal

Build the visible user experience and connect the chatbot to an AI provider.

By the end of Week 3, users should be able to open the app, access the chatbot, submit appointment requests, and create reminders.

What to Build

Pages

1. Home page
2. Chat page
3. Appointment booking page
4. Reminder page
5. About page

Chat Interface

Features:

1. Message bubbles
2. User input field
3. Send button
4. Loading indicator
5. Suggested prompts
6. Safety disclaimer
7. Chat history

Suggested Patient Prompts

1. I have a headache. What should I consider?
2. How do I prepare for a clinic visit?

3. When should I seek urgent medical care?
 4. Can I book an appointment?
 5. Can you remind me to take my medication?
-

AI Safety Rules

The chatbot should:

1. Provide general health guidance only
2. Avoid final diagnosis
3. Avoid prescribing medication
4. Encourage users to consult healthcare professionals
5. Escalate emergency symptoms clearly
6. Keep responses simple and patient-friendly
7. Avoid collecting unnecessary sensitive information

Sample System Prompt

You are AfyaFlow AI, a healthcare support assistant.

Your role is to provide general health guidance, support appointment booking, and help users understand when they should seek professional medical care.

You must not diagnose diseases, prescribe medication, or replace a licensed healthcare professional.

If a user mentions emergency symptoms such as severe chest pain, difficulty breathing, severe bleeding, loss of consciousness, stroke-like symptoms, severe allergic reaction, or danger in pregnancy, advise them to seek urgent medical care immediately.

Keep responses simple, calm, respectful, and easy to understand.

AI Integration Design

Create a dedicated service file for AI interaction.

```
apps/chatbot/services/ai_service.py
```

This keeps the AI provider separate from the rest of the app.

Benefits:

1. Easier to switch from one AI provider to another

2. Cleaner code
 3. Easier testing
 4. Better control over prompts
 5. Better safety handling
-

Claude Code Tasks for Week 3

Use Claude Code to:

1. Generate UI templates
2. Improve Tailwind styling
3. Build the chat form interaction
4. Create the AI service layer
5. Add safety checks
6. Write tests for chatbot behavior
7. Review the user flow
8. Improve error messages

Example Prompt

Create a Django template for the AfyaFlow AI chat page using Tailwind CSS. Include message bubbles, a user input box, suggested health support questions, a loading state, and a visible safety disclaimer.

Example Prompt

Create an AI service layer for the chatbot app. The service should accept a user message, apply the healthcare safety system prompt, call the AI provider, and return a safe response. Keep provider-specific logic isolated in one file.

LinkedIn Post for Week 3

Post Theme

Building the chatbot experience and learning responsible AI design.

Sample LinkedIn Post

Week 3 of building AfyaFlow AI.

This week, the product started to feel real.

I worked on the user interface and the AI chatbot experience.

Key progress:

- Home page
- Chat page
- Appointment booking page
- Reminder page
- Basic chatbot interface
- AI service layer
- Safety prompt design

The most important lesson this week was that healthcare AI needs boundaries.

The chatbot should not behave like a doctor. It should provide general guidance, support simple patient questions, and encourage users to seek professional care when needed.

For AfyaFlow AI, I'm designing the assistant to:

- Avoid diagnosis
- Avoid prescriptions
- Explain limitations clearly
- Escalate emergency symptoms
- Keep responses simple and safe

I'm also using Claude Code to move faster, especially for UI scaffolding, service layer design, test generation, and code review.

Next step: testing, deployment, and preparing the final MVP demo.

#BuildInPublic #AI #HealthTech #Django #Python #ResponsibleAI #SoftwareEngineering

8. Week 4: Testing, Deployment and Final Demo

Goal

Polish the MVP, test the core flows, deploy the application, and share the final build publicly.

What to Complete

1. Chatbot flow
2. Appointment booking

3. Reminder creation
 4. Admin dashboard review
 5. Error handling
 6. Mobile responsiveness
 7. README update
 8. Environment variable documentation
 9. Deployment
 10. Final demo video
-

Testing Checklist

Area	What to Test
Chatbot	Does it respond safely and clearly?
Safety	Does it avoid diagnosis and prescriptions?
Emergency Guidance	Does it guide urgent cases to seek care?
Appointment Form	Does it submit correctly?
Reminder Form	Does it save reminder details?
Database	Are records saved in PostgreSQL?
Admin	Can records be viewed in Django admin?
API	Do endpoints return correct responses?
Mobile	Does the UI work on phone screens?
Deployment	Does the live app work end to end?

Deployment Checklist

1. Set environment variables
2. Configure PostgreSQL database
3. Run migrations

4. Create superuser
 5. Collect static files
 6. Deploy app
 7. Test live URLs
 8. Update README
 9. Add live demo link to GitHub
 10. Prepare LinkedIn demo post
-

Claude Code Tasks for Week 4

Use Claude Code to:

1. Review the entire codebase
2. Find bugs
3. Generate missing tests
4. Improve README
5. Create deployment checklist
6. Review environment variable setup
7. Suggest refactoring
8. Prepare release notes

Example Prompt

Review this Django project for MVP readiness. Check for missing environment variables, weak error handling, untested flows, unclear README steps, and deployment risks. Give me a prioritized fix list before editing anything.

Example Prompt

Write a clean README for this project. Include project overview, features, tech stack, setup steps, environment variables, database migration steps, running tests, deployment notes, and future improvements.

LinkedIn Post for Week 4

Post Theme

Completing and deploying the AfyaFlow AI MVP.

Sample LinkedIn Post

I've completed the first MVP of AfyaFlow AI.

Over the past few weeks, I have been building an AI-powered healthcare assistant using Python, Django, PostgreSQL, and AI API integration.

The MVP includes:

- Patient chatbot interface
- General health guidance
- Appointment booking
- Reminder support
- PostgreSQL database
- Django admin dashboard
- AI service layer
- Safety prompt design
- Deployed demo

This project taught me that building an AI product is not just about calling an AI API.

It requires:

- Clear product requirements
- Good solution design
- Thoughtful database structure
- Safe user experience
- Testing
- Deployment discipline
- Clear documentation

One of my biggest lessons was the importance of designing boundaries for healthcare AI. The assistant must be helpful, but it must also be clear that it does not replace professional medical care.

Next improvements:

- User authentication
- Clinic admin dashboard
- SMS reminders
- Email reminders
- Better triage flows
- Analytics dashboard

GitHub:

[Add link]

Live Demo:

[Add link]

#BuildInPublic #Django #Python #PostgreSQL #AI #HealthTech #SoftwareEngineering

9. LinkedIn Content Calendar

Recommended Posting Frequency

Three posts per week is a strong rhythm.

Suggested days:

- 1. Monday: Build update
- 2. Wednesday: Technical lesson
- 3. Friday: Reflection, demo, or mistake learned

4-Week LinkedIn Calendar

Week	Day	Post Theme	Suggested Asset
Week 1	Monday	Introducing AfyaFlow AI	Product idea graphic
Week 1	Wednesday	Why I started with a PRD	PRD screenshot
Week 1	Friday	Solution Design Document lessons	Architecture sketch
Week 2	Monday	Django project setup	GitHub screenshot
Week 2	Wednesday	PostgreSQL model design	Database diagram
Week 2	Friday	Building APIs with Django REST Framework	API screenshot
Week 3	Monday	Designing the chatbot UI	Chat UI screenshot
Week 3	Wednesday	Healthcare AI safety rules	Prompt snippet
Week 3	Friday	Connecting the AI service layer	Code screenshot
Week 4	Monday	Testing the MVP	Testing checklist

Week 4	Wednesday	Deployment lessons	Live app screenshot
Week 4	Friday	Final MVP demo	Demo video

10. LinkedIn Content Strategy for Securing an Entry-Level Developer Role

Strategic Intent

The purpose of the LinkedIn journey is not only to document progress. The real goal is to create visible proof that she is ready for an entry-level software developer role.

Every post should help a recruiter, hiring manager, founder, or senior developer understand one of five things:

1. She can think through a product problem
2. She can translate requirements into software
3. She can write and explain code
4. She can debug and learn independently
5. She can complete and ship a working project

The posts should avoid sounding like random learning updates. They should be structured as a public portfolio campaign.

Target Positioning

Use this positioning throughout the LinkedIn journey:

I am building AfyaFlow AI in public to demonstrate my software engineering, backend development, AI integration, and product-building skills through a real-world HealthTech MVP.

This makes the intent clear without over-explaining that she is looking for an entry-level role in every post.

Profile Positioning Before Posting

Before the posting journey starts, her LinkedIn profile should be aligned to the role she wants.

Suggested Headline

Software Developer | Python, Django, PostgreSQL | Building AI-Powered HealthTech Products

Alternative:

Entry-Level Software Developer | Python, Django, PostgreSQL | AI and HealthTech Projects

About Section Direction

The About section should clearly mention:

1. She is focused on software development
2. Her core stack is Python, Django, PostgreSQL, HTML, CSS, JavaScript, and REST APIs
3. She is building AfyaFlow AI as a portfolio project
4. She is interested in backend, full-stack, HealthTech, and AI-enabled applications
5. She is open to entry-level developer opportunities, internships, apprenticeships, and junior software roles

Featured Section

Add the following once ready:

1. GitHub repository
2. Live demo link
3. Final AfyaFlow AI demo post
4. PRD or project case study
5. Short demo video

Content Rules

Every post should include at least one of the following:

1. A screenshot
2. A code snippet
3. A GitHub commit
4. A diagram
5. A lesson learned
6. A short demo video
7. A problem and how it was solved

8. A link to the project or repository where appropriate

Avoid posts that only say:

Today I learned Django.

Use posts that say:

Today I used Django models to design how appointment requests will be stored in AfyaFlow AI.

This makes the learning tied to a real product.

11. Hiring-Focused Content Pillars

Pillar 1: Product Thinking

Purpose: Show that she does not just code blindly. She understands users, requirements, and scope.

Post examples:

1. Why I started AfyaFlow AI with a Product Requirements Document
2. How I defined the MVP scope before writing code
3. What I included and excluded from version 1
4. Why healthcare AI needs clear product boundaries
5. How user journeys shaped my database design

Recruiter signal:

This person can understand requirements and build with purpose.

Pillar 2: Backend Engineering

Purpose: Show hands-on technical capability with Python, Django, PostgreSQL, and APIs.

Post examples:

1. How I structured my Django project into apps
2. How I designed the Appointment model in PostgreSQL
3. How I created API endpoints with Django REST Framework

4. How I handled validation in appointment booking
5. How I used Django admin to manage MVP data

Recruiter signal:

This person can work on real backend tasks.

Pillar 3: Problem-Solving and Debugging

Purpose: Show resilience and independent learning.

Post examples:

1. A bug I faced while connecting Django to PostgreSQL
2. How I fixed a migration error
3. What I learned from a broken API response
4. How I debugged a form submission issue
5. Why error messages are part of the learning process

Recruiter signal:

This person can troubleshoot and keep going.

Pillar 4: AI Integration and Responsible AI

Purpose: Show modern capability while being thoughtful about safety.

Post examples:

1. How I designed the AI service layer for AfyaFlow AI
2. Why I separated AI logic from Django views
3. How I created a safety prompt for healthcare guidance
4. Why the chatbot should not diagnose or prescribe
5. What I learned about responsible AI product design

Recruiter signal:

This person understands how to integrate AI responsibly.

Pillar 5: Proof of Work and Shipping

Purpose: Show that she can complete projects, not only start them.

Post examples:

1. My first working chatbot flow
2. Appointment booking is now saving to PostgreSQL
3. Testing the reminder creation flow
4. AfyaFlow AI is now deployed
5. Final MVP demo and lessons learned

Recruiter signal:

This person can ship working software.

12. Recommended Posting Rhythm

Recommended Frequency

The recommended posting frequency is three posts per week.

This is the right balance because it creates consistency, visibility, and proof of progress without becoming overwhelming.

The weekly structure should be:

Day	Post Type	Purpose	Hiring Signal
Monday	Build Update	Show what was built or started	Consistency and execution
Wednesday	Technical Learning	Explain a concept, decision, or implementation	Technical understanding
Friday	Proof, Reflection, or Demo	Share evidence, lesson learned, fix, screenshot, or demo	Ability to solve problems and ship

The rule for every week is:

- 1 post for progress
- 1 post for technical learning
- 1 post for proof of work

This structure makes the journey intentional for securing an entry-level developer role because it shows she can plan, build, explain, debug, and ship.

Monday: Build Update

Monday posts should show progress.

The post should answer:

What did I build, start, complete, or improve?

Examples:

1. Started AfyaFlow AI
2. Completed the Product Requirements Document
3. Created the Solution Design Document
4. Set up the Django project
5. Connected PostgreSQL
6. Created appointment and reminder models
7. Built the chatbot page
8. Deployed the MVP

Suggested structure:

This week, I worked on [specific deliverable].

This matters because [why it is important to the product].

What I completed:

- Item 1
- Item 2
- Item 3

Next, I will work on [next step].

Wednesday: Technical Learning

Wednesday posts should show depth.

The post should answer:

What technical concept did I learn or apply?

Examples:

1. Why I chose Django for AfyaFlow AI
2. How Django models map to PostgreSQL tables
3. How Django REST Framework serializers work
4. Why database design starts from the user journey
5. Why I created a separate AI service layer
6. How environment variables protect sensitive configuration
7. How migrations help manage database changes
8. Why validation matters in appointment booking

Suggested structure:

One thing I learned while building AfyaFlow AI is [technical lesson].

I applied it when working on [specific feature].

Before this, I thought [old understanding].

Now I understand [new understanding].

This will help me build better software because [practical value].

Friday: Proof, Reflection, or Demo

Friday posts should show evidence.

The post should answer:

What can I show as proof of progress?

Examples:

1. Screenshot of the PRD
2. Architecture diagram
3. GitHub commit screenshot
4. Code snippet
5. Database model screenshot
6. API response screenshot
7. Short chatbot demo video
8. Bug and fix explanation
9. Final MVP walkthrough

Suggested structure:

Small AfyaFlow AI progress update.

This week, I worked on [feature or problem].

Here is what is now working:

- Item 1
- Item 2
- Item 3

One challenge I faced was [challenge].
I solved it by [solution].

Next improvement: [next step].

4-Week 3-Posts-Per-Week Plan

Week	Monday: Build Update	Wednesday: Technical Learning	Friday: Proof, Reflection, or Demo
Week 1	Introducing AfyaFlow AI and setting the build goal	Why I started with a Product Requirements Document	Solution Design Document and first architecture sketch
Week 2	Setting up Django and PostgreSQL	How Django models translate product requirements into database structure	First API endpoint or Django admin screenshot
Week 3	Building the chatbot interface	Designing the AI service layer and healthcare safety prompt	Working chatbot demo or debugging lesson
Week 4	Testing the core MVP flows	Deployment lessons and environment setup	Final MVP demo with GitHub, live link, and entry-level role CTA

Weekly Execution Checklist

Before each post, confirm that it has at least three of the following:

1. Clear progress update
2. Specific technical detail
3. Screenshot, diagram, code snippet, or demo
4. Lesson learned
5. Next step

6. Relevant hashtags
7. Professional tone

A strong post should not only say what happened. It should show what was built, why it matters, what was learned, and what comes next.

Recommended Hashtag Set

Use 4 to 7 hashtags per post.

Core hashtags:

#BuildInPublic #Python #Django #PostgreSQL #SoftwareDevelopment #HealthTech #AI

For backend posts:

#BackendDevelopment #DjangoRESTFramework #APIDesign #PostgreSQL

For job-focused posts:

#SoftwareEngineering #EntryLevelDeveloper #TechCareers #WomenInTech

For AI posts:

#AI #ResponsibleAI #HealthTech #AIDevelopment

Important Posting Principle

Do not make every post sound like a job request.

The job search should be implied through quality, consistency, proof of work, and clear positioning.

Use a direct job-search call to action mainly on milestone posts, especially the final MVP demo post.

Example:

I'm open to entry-level software developer roles, internships, apprenticeships, and backend or full-stack opportunities where I can contribute using Python, Django, PostgreSQL, and AI-enabled development.

13. 4-Week Hiring-Focused LinkedIn Calendar

Week	Day	Post Type	Topic	Hiring Signal
Week 1	Monday	Positioning	Starting AfyaFlow AI	Product ambition
Week 1	Wednesday	Product Thinking	Why I wrote a PRD first	Requirements thinking
Week 1	Friday	Solution Design	How I planned the architecture	Technical planning
Week 2	Monday	Backend Build	Setting up Django and PostgreSQL	Backend foundation
Week 2	Wednesday	Database Design	Designing the Appointment model	Data modeling
Week 2	Friday	API Build	Creating the first DRF endpoint	API capability
Week 3	Monday	UI Build	Building the chatbot interface	Frontend awareness
Week 3	Wednesday	AI Integration	Creating the AI service layer	AI capability
Week 3	Friday	Debugging	A bug I fixed this week	Problem-solving
Week 4	Monday	Testing	Testing the core user flows	Quality mindset
Week 4	Wednesday	Deployment	Deploying the MVP	Shipping ability

14. Post Structure Template

Each post should follow this structure:

1. Hook

Start with what she built, learned, or solved.

Example:

This week, I turned the AfyaFlow AI product requirements into my first Django data models.

2. Context

Explain the product or technical reason.

Example:

Because the app needs to support appointment booking, I needed a clean way to store patient appointment requests in PostgreSQL.

3. What I Built

List the specific work done.

Example:

I worked on:

- Appointment model
- Status choices
- Admin registration
- Database migration
- Basic API serializer

4. What I Learned

Share one real lesson.

Example:

The biggest lesson was that model design forces you to think carefully about the real-world workflow before writing too much code.

5. Proof

Add screenshot, code snippet, diagram, GitHub commit, or short demo.

6. Soft Call to Action

End with a professional close.

Example:

Next, I'll connect this to the appointment booking API and test the full flow.

15. Sample Hiring-Focused Posts

Post 1: Project Launch

I'm starting a new software build journey: AfyaFlow AI.

AfyaFlow AI is an AI-powered healthcare assistant designed to support routine patient guidance, appointment booking, and reminders.

I'm building this project to strengthen and demonstrate my skills in:

- Python
- Django
- PostgreSQL
- REST APIs
- AI integration
- Product documentation
- Deployment

Before writing too much code, I'm starting with two key documents:

- Product Requirements Document
- Solution Design Document

The goal is to build this like a real software product, from requirements to architecture, implementation, testing, and deployment.

I'll be documenting the journey here with progress updates, technical lessons, mistakes, fixes, and demos.

#BuildInPublic #Python #Django #PostgreSQL #SoftwareDevelopment #AI #HealthTech

Post 2: PRD Post

Before writing code for AfyaFlow AI, I started with a Product Requirements Document.

The PRD helped me clarify:

- Who the product is for
- What problem it solves
- What features belong in the MVP
- What should stay out of version 1
- What success looks like

For this MVP, I'm focusing on:

- Patient chatbot
- Appointment booking
- Reminder creation
- Basic admin dashboard
- Safe AI guidance

I also defined what the product should not do.

For example, AfyaFlow AI should not diagnose, prescribe medication, or replace a healthcare professional.

This was an important lesson for me: good software starts before the first line of code.

#ProductDevelopment #SoftwareEngineering #Django #HealthTech #AI

Post 3: Solution Design Post

This week, I created the Solution Design Document for AfyaFlow AI.

The goal was to think through the technical structure before building the MVP.

The planned stack is:

- Python

- Django
- Django REST Framework
- PostgreSQL
- Tailwind CSS
- AI API integration

I also mapped the main modules:

- Accounts
- Chatbot
- Appointments
- Reminders
- Core

One thing I'm learning is that architecture does not have to be complex to be useful.

For this stage, the goal is a simple structure that is clean, understandable, and easy to extend.

Next, I'll start setting up the Django project and database models.

#SoftwareArchitecture #Django #Python #PostgreSQL #BuildInPublic

Post 4: Backend Build Post

I've started building the backend foundation for AfyaFlow AI.

This week, I worked on the Django project setup and PostgreSQL configuration.

Progress so far:

- Created the Django project
- Set up the main apps
- Connected PostgreSQL
- Created the first models
- Ran initial migrations
- Registered models in Django admin

The most useful lesson was understanding how Django apps help separate responsibilities.

Instead of putting everything in one place, I'm separating the project into apps for chatbot, appointments, reminders, accounts, and core functionality.

This is helping me keep the codebase cleaner as the project grows.

#Django #Python #PostgreSQL #BackendDevelopment #SoftwareEngineering

Post 5: Debugging Post

Today's AfyaFlow AI lesson: debugging is part of building.

While connecting Django to PostgreSQL, I ran into an issue with database configuration and migrations.

What I checked:

- Database name
- User permissions
- Environment variables
- Django settings
- Migration files

The issue reminded me that backend development is not only about writing models and views. It is also about understanding how the application, environment, and database connect.

Once I fixed it, the migrations ran successfully and the models appeared in Django admin.

Small win, but a useful one.

#Django #PostgreSQL #Debugging #BackendDevelopment #BuildInPublic

Post 6: AI Integration Post

This week, I started working on the AI chatbot layer for AfyaFlow AI.

Instead of placing AI logic directly inside the Django view, I created a separate service layer.

Why?

Because separating the AI logic makes it easier to:

- Change AI providers later
- Manage prompts safely
- Test chatbot behavior
- Keep the Django views cleaner
- Improve the system over time

For healthcare use cases, I'm also adding clear safety boundaries.

The assistant should provide general guidance, but it should not diagnose, prescribe medication, or replace a healthcare professional.

This part of the project is teaching me that AI integration is not just a technical task. It is also a product and safety design task.

#AI #Django #Python #HealthTech #ResponsibleAI #SoftwareEngineering

Post 7: Final Demo Post

I've completed the first MVP of AfyaFlow AI.

AfyaFlow AI is an AI-powered healthcare assistant built with Python, Django, PostgreSQL, and AI API integration.

The MVP includes:

- Patient chatbot interface
- General healthcare guidance
- Appointment booking
- Reminder creation
- PostgreSQL database
- Django admin dashboard
- AI service layer
- Safety prompt design
- Deployed demo

This project helped me practice the full software development process:

- Product requirements
- Solution design
- Backend development
- Database modeling
- API design
- AI integration
- Testing
- Deployment
- Documentation

The biggest lesson: building a useful AI product requires more than connecting to an API. It requires clear requirements, thoughtful design, safety boundaries, testing, and iteration.

GitHub:

[Add link]

Live Demo:

[Add link]

#Python #Django #PostgreSQL #AI #HealthTech #SoftwareDevelopment #BuildInPublic

16. Engagement Strategy

Posting alone is not enough. She should also engage intentionally.

Each week, she should comment on posts by:

1. Software engineers
2. Engineering managers
3. Tech recruiters
4. Startup founders
5. HealthTech builders
6. Python and Django developers

Recommended comment style:

This is helpful. I'm currently building a Django and PostgreSQL HealthTech MVP, and I'm seeing the same thing around the importance of clean requirements before implementation.

This creates visibility without sounding like she is begging for a job.

17. Job Search Call to Action Strategy

Do not add "I am looking for a job" in every post.

Use it strategically in selected posts.

Soft CTA

I'm continuing to build and share my progress as I prepare for software development opportunities where I can contribute, learn, and grow.

Clear CTA for Final Demo Post

I'm open to entry-level software developer roles, internships, apprenticeships, and backend or full-stack opportunities where I can contribute using Python, Django, PostgreSQL, and AI-enabled development.

Direct Networking CTA

If you are a developer, hiring manager, founder, or recruiter working around Python, Django, HealthTech, or AI products, I'd be happy to connect and learn from your experience.

18. GitHub and LinkedIn Alignment

Her LinkedIn posts should point people to proof.

Her GitHub should include:

1. Clean README
2. Project overview
3. Features list
4. Tech stack
5. Screenshots
6. Setup instructions
7. Environment variable guide
8. Database migration steps
9. Demo link
10. Future improvements

Her LinkedIn should include:

1. Featured GitHub repo
2. Featured live demo
3. Featured final MVP post
4. Clear headline
5. Clear About section

This creates a complete hiring funnel:

LinkedIn post → Profile → Featured project → GitHub → Live demo → Interview conversation

19. Content Pillars

1. Product Thinking

Topics:

1. Why PRDs matter
2. How user journeys guide development
3. How to define MVP scope
4. Why healthcare AI needs clear boundaries

2. Technical Execution

Topics:

1. Django project setup
2. PostgreSQL data modeling
3. Django REST Framework APIs
4. AI service layer
5. Deployment

3. AI Productivity

Topics:

1. Using Claude Code to plan features
2. Using Claude Code to review code
3. Using Claude Code to generate tests
4. Using AI without losing understanding
5. How AI improves speed when requirements are clear

4. Lessons Learned

Topics:

1. Debugging lessons
2. Mistakes made
3. What changed from the original plan
4. What was harder than expected
5. What the next version will improve

5. Demo and Proof of Work

Topics:

1. Screenshots
2. GitHub commits
3. Short videos
4. Architecture diagrams
5. Before and after improvements

20. Better LinkedIn Positioning

The positioning should be professional and confident.

Avoid saying:

I am just a junior developer trying to learn.

Use:

I am building AfyaFlow AI in public to strengthen my software engineering, product thinking, and AI development skills through a real-world HealthTech project.

Avoid saying:

This is a small practice project.

Use:

This is an MVP designed to explore how AI can support routine patient engagement and clinic workflow efficiency.

Avoid saying:

I used AI to write the code for me.

Use:

I used Claude Code as a development accelerator for planning, scaffolding, debugging, testing, and documentation while reviewing and understanding each implementation decision.

12. Final Recommended Build Narrative

The best story is not just:

I built a chatbot.

The stronger story is:

I built an AI-powered healthcare assistant from product requirements to solution design, backend development, database design, AI integration, testing, and deployment.

That story shows:

1. Product thinking
 2. Software engineering discipline
 3. AI awareness
 4. Documentation skills
 5. Ability to ship
 6. Professional communication
-

13. Final MVP Success Criteria

By the end of the build, AfyaFlow AI should have:

1. A clear PRD
 2. A clear Solution Design Document
 3. A working Django project
 4. PostgreSQL database setup
 5. Chatbot interface
 6. Appointment booking flow
 7. Reminder creation flow
 8. AI safety prompt
 9. Django admin view
 10. README documentation
 11. GitHub repository
 12. Deployed live demo
 13. Final LinkedIn demo post
-

14. Suggested Future Enhancements

After the MVP, the next version can include:

1. User authentication
2. Clinic admin dashboard
3. SMS reminders
4. Email reminders
5. Appointment status management
6. Patient profile management
7. Multilingual support
8. Basic triage decision trees
9. Analytics dashboard
10. Role-based access control
11. Audit logs
12. Integration with clinic management systems

15. Overall Recommendation

The strongest version of this journey is:

Building AfyaFlow AI in public: a Django, PostgreSQL, and AI-powered healthcare assistant, taken from PRD and Solution Design to deployed MVP.

This gives the project a clear professional identity.

It also helps the builder stand out because she will not only show code. She will show how she thinks, plans, builds, tests, documents, and ships a real product.